

Trabalho Banco de dados

Agnaldo dos Santos Prieto Junior

GRR20221259

Mateus Garcia Baiak

GRR20224378

Departamento de Informática

Universidade Federal do Paraná – UFPR

Curitiba, Brasil

aspj22@inf.ufpr.br

mgb22@inf.ufpr.br

I. INTRODUÇÃO

Esse trabalho tem como objetivo implementar, em linguagem C, uma ferramenta de linha de comando (fdtool) para manipular Dependências Funcionais (DFs) em esquemas relacionais com atributos no intervalo A..Z.

Foram realizados os seguintes cálculos:

- fecho X^+ de um conjunto de atributos $X \subseteq U$ dado F;
- Cobertura Mínima de F (RHS unitário, remoção de atributos estranhos e DFs redundantes);
- Chaves candidatas (todas as mínimas K tais que $K^+ = U$);
- Verificação de formas normais: BCNF e 3FN.

II. ESTRUTURAS

Foram utilizadas algumas estruturas de dados no desenvolvimento do trabalho:

- Dependencia: estrutura contendo os atributos à esquerda e direita das dependências funcionais;
- listaDependencias: contém a estrutura Dependência e a quantidade de dependências;
- chaves: estrutura contendo as chaves candidatas e a quantidade;
- lista: estrutura contendo o início da lista, a quantidade e os atributos. Utilizada na aplicação da Busca em Largura para descobrir as chaves candidatas.

III. FECHO

A função começa passando o atributo que deve ser calculado o fecho para um vetor *fecho*, depois passa pela DF verificando se o atributo do lado esquerdo está no vetor fecho, se sim, adiciona os atributos do lado direito no vetor.

Por fim é realizada uma ordenação dos atributos usando *qsort* e retorna o vetor.

A. Complexidade

A: quantidade máxima de atributos;

L: tamanho máximo de um lado da DF;

F: número de DFs.

O algoritmo roda um loop externo que pode repetir até A vezes, porque a cada iteração no máximo 1 atributo novo entra no fecho. O algoritmo roda um loop externo que pode repetir até A vezes, porque a cada iteração no máximo 1 atributo novo entra no fecho. Para cada DF:

- verifica cada atributo da esquerda e compara com cada atributo já no fecho;
- para cada atributo da direita compara novamente com cada atributo do fecho;

Portanto, a complexidade é $O(F * L * A^2)$.

IV. COBERTURA MÍNIMA

A função responsável pela Cobertura Mínima da DFs realiza por três etapas:

A. Etapa 1

A primeira etapa é tornar o lado direito de cada dependência funcional unitário. No código é alocada uma estrutura Dependencia *DF_unitario*, onde no laço passando por todas as DFs é armazenado no *DF_unitario* o lado direito da DF e cada atributo do lado direito.

B. Etapa 2

A segunda etapa é remover atributos estranhos do lado esquerdo. O código vai reduzindo a quantidade de atributos no lado esquerdo e verificando, através do fecho, se ainda alcança os atributos do lado direito do *DF_unitario* sem estar reduzida. Se alcança, então o lado esquerdo do *DF_unitario* é atualizado.

C. Etapa 3

A terceira e última etapa é remover dependências funcionais redundantes. O código aloca uma estrutura *DF_final*, cria uma estrutura temporária e entra em um laço passando por todas as DFs unitárias, a cada passo do laço é retirado temporariamente uma DF e calculado o fecho do lado esquerdo dessa DF, se o atributo do lado direito dessa DF for encontrada no fecho, então essa DF pode ser removida.

D. Complexidade

A: quantidade máxima de atributos;
L: tamanho máximo de um lado da DF;
F: número de DFs.

Na etapa 1: Percorre todas as FDs e todos os atributos da direita. Logo, a complexidade é $O(F*L)$.

Na etapa 2: Para cada atributo da esquerda:

- monta uma versão reduzida da esquerda: $O(L)$;
- copia todas as dependências: $O(F)$;
- chama fecho: $O(F*L*A^2)$;
- compara fecho com lado direito: $O(A)$;

Portanto, a complexidade é $O(F^2*L^2*A^2)$.

V. CHAVES CANDIDATAS

Para o cálculo das chaves candidatas é utilizado um algoritmo de busca em largura, para o funcionamento dela temos a biblioteca listaDeBusca que contém as funções básicas para uma lista como: criar a lista, insere um elemento na lista, devolve um elemento da lista.

É criada uma lista e adicionado todos os atributos no início dela, e então entramos em um loop que trabalha da seguinte forma: removemos o primeiro elemento da lista, verificamos se ele é chave e se for verificamos se ele já não está incluído em uma chave já existente, se passar nessa verificação ele é salvo em uma struct chave, se o item verificado não for chave adicionamos na lista as permutações do item com os outros atributos, esse loop continua até não termos mais itens na lista e ao final os devolvemos as chaves.

A. Complexidade

A: quantidade máxima de atributos;
L: tamanho máximo de um lado da DF;
F: número de DFs.

O BFS gera até 2^A subconjuntos de atributos e também é calculado o fecho de atributos, logo a complexidade é $O(2^A * F * L * A^2)$.

VI. BCNF E 3FN

A função começa pegando os atributos primos das chaves candidatas, seguindo para a verificar se a lista está em BCNF, ou seja, verifica se o lado esquerdo de cada DF é uma superchave, se verdadeiro, então está em BCNF.

Posteriormente é feita a verificação da 3FN, repete a verificação se a chave é uma superchave, se não for, é feita a verificação se os atributos do lado direito são primos, se verdadeiro, então está em 3FN.

A. Complexidade

Encontrar as chaves candidatas exige testar todos os subconjuntos de atributos: 2^A .

Para cada subconjunto é calculado o fecho: $O(F*L*A^2)$.

Logo, a complexidade é $O(2^A * F * L * A^2)$.

VII. ESTUDOS DE CASO

A. Caso 1

$U = \{A, B, C, D\}$

$F = \{ AB \rightarrow CD, A \rightarrow C, C \rightarrow D \}$

1) *Fecho*: Calculando o fecho de A, vai por A no vetor *fecho* e posteriormente entrar em um laço passando por cada DF. Nesse laço é verificado a DF $AB \rightarrow CD$, como B não está no fecho, então os elementos à direita não adicionados ao vetor *fecho*. Depois é verificado $A \rightarrow C$, A está no vetor *fecho*, logo C é adicionado ao fecho de A. Por fim vemos $C \rightarrow D$, como C está no vetor *fecho*, D é adicionado.

Dessa forma, o fecho de A é ACD.

Seguindo o algoritmo, se executar para B obterá apenas B, para C terá CD e D apenas D.

2) *Cobertura Mínima*: Calculando a cobertura mínima, primeiro é preciso tornar o lado direito unitário, será apenas necessário no caso $AB \rightarrow CD$, separando em $AB \rightarrow C$ e $AB \rightarrow D$.

Resultado após etapa 1: $\{AB \rightarrow C, AB \rightarrow D, A \rightarrow C, C \rightarrow D\}$

A segunda etapa é remover atributos estranhos do lado esquerdo. Apenas $AB \rightarrow C$ e $AB \rightarrow D$ serão testados. Em $AB \rightarrow C$, removendo A e calculando o fecho de B, como C não está em B^+ , então não pode remover A. Removendo B e calculando A^+ , vemos que C está presente no fecho, logo B é atributo estranho e a DF passa de $AB \rightarrow C$ para $A \rightarrow C$. O mesmo ocorre executando em $AB \rightarrow D$, tornando $AB \rightarrow D$ para $A \rightarrow D$.

Resultado após etapa 2: $\{A \rightarrow C, A \rightarrow D, C \rightarrow D\}$

A terceira etapa é remover dependências redundantes. $A \rightarrow C$ é removida, calcula o fecho de A sendo A, D, como C não aparece, a DF não pode ser removida.

Agora $A \rightarrow D$ é removida, calculando o fecho de A, D é encontrado, logo a DF é redundante e pode ser removida.

Por fim, testando para $C \rightarrow D$, é verificado que não é redundante, mantendo a DF.

Assim, a cobertura mínima final é $\{A \rightarrow C, C \rightarrow D\}$.

3) *Chaves Candidatas*: É realizada a busca em largura para encontrar as chaves candidatas. Os atributos A, B, C e D serão enfileirados e será realizado o cálculo do fecho para cada um.

Calculando o fecho de A, notasse que $A^+ \neq U$, logo não é chave, gera filhos (atributos que ainda não estão em A): adiciona "AB", "AC", "AD" à fila.

Seguindo iterando pela lista até chegar em AB, calculando o fecho vemos que AB é superchave, salvando AB na estrutura de chaves. Sabendo que AB é chave, então não será mais expandido.

Seguindo iterando pela lista não é possível encontrar mais chaves candidatas, logo a única chave candidata encontrada é AB

4) *BCNF e 3FN*: O algoritmo começa pegando as chaves candidatas e os atributos primos a partir dessas chaves.

Depois começa a verificar se está em BCNF, conferindo se todos do lado esquerdo da DF são superchave. Como apenas AB é superchave, tem violação da BCNF em $A \rightarrow C$.

Agora é verificar se está em 3FN, onde é preciso verificar se os atributos do lado direito são primos. Tem violação em 3FN no caso $A \rightarrow C$, em que C não é primo.

B. Caso 2

$$U = \{A, B, C\}$$
$$F = \{A \rightarrow B, B \rightarrow A\}$$

1) *Fecho*: Calculando o fecho de A, o próprio A é adicionado ao vetor fecho e posteriormente entrar em um laço passando por cada DF. No primeiro laço verificamos a DF $A \rightarrow B$, A está no fecho então B é adicionado no fecho uma vez que ele não estava anteriormente. Depois verificamos para $B \rightarrow A$, B está no fecho então A seria adicionado no fecho, como A já faz parte do fecho ele não será adicionado.

Ao final do laço é verificado que houve mudanças, então o laço será executado mais uma vez, dessa segunda iteração não terá nenhuma alteração e o código irá devolver AB.

2) *Cobertura Mínima*: Ao iniciar o calculo da cobertura mínima verificamos que as DFs já são unitárias, a segunda etapa removemos atributos estranhos do lado esquerdo, como cada DF tem apenas um atributo do lado esquerdo mantemos as DFs da mesma forma, então removemos as duplicatas que no nosso caso não temos nenhuma. Por fim verificamos as dependências redundantes e vemos que não precisamos remover nenhuma das DFs, Assim a cobertura mínima fica $\{A \rightarrow B, B \rightarrow A\}$, igual a original.

3) *Chaves Candidatas*: É acionado a lista os atributos A, B, C no início do algoritmo, removemos A verificamos que não é chave e adicionamos na lista AB, AC, recomeçamos o loop e retiramos B e depois na próxima iteração C e o mesmo é feito para os dois sem adicionar elementos repetidos na lista, então temos a lista com: $\{AB, AC, BC\}$, retiramos AB e calculamos seu fecho, ele não é chave então não é salvo na struct chaves, e adicionamos ABC a lista, retiramos AC que é chave e não adicionamos ninguém já que qualquer filho não seria minimal, removemos BC que é feito o mesmo processo de AC, e por fim verificamos ABC que tem AC e BC como chave dentro dele então não é minimal.

A lista ficou vazia então terminamos o laço, e ao fim de tudo temos as chaves: AC, BC.

4) *BCNF e 3FN*: Na verificação de BCNF tem violação em $A \rightarrow B$, pois A não é superchave, e não tem violação em 3FN, já que todos os atributos do lado direito das DFs são primos.

C. Caso 3

$$U = \{A, B, C\}$$
$$F = \{A \rightarrow B, A \rightarrow C\}$$

1) *Fecho*: Calculando o fecho de A, o próprio A é adicionado ao vetor fecho e posteriormente entrar em um laço passando por cada DF. No primeiro laço verificamos a DF $A \rightarrow B$, A está no fecho então B é adicionado no fecho uma vez que ele não estava anteriormente. O mesmo ocorre para $A \rightarrow C$ então ao final temos que o fecho será ABC.

2) *Cobertura Mínima*: Começamos verificando que as dependências já são unitária, depois removemos elementos estranhos do lado esquerdo o que também não resulta em alteração, vemos se temos duplicatas, o que não temos. Por fim removemos dependências redundantes, verificamos que nenhuma das DFs é redundante então não fazemos nenhuma remoção. Por fim verificamos as dependências redundantes e vemos que não precisamos remover nenhuma das DFs, Assim a cobertura mínima fica $\{A \rightarrow B, A \rightarrow C\}$, igual a original.

3) *Chaves Candidatas*: É acionado a lista os atributos A, B, C no início do algoritmo, removemos A verificamos que é chave, não adicionamos nenhum filho na lista e vamos para a próxima iteração, removemos B e na próxima iteração C para os quais não são chaves, ficamos com a lista dessa forma: $\{AB, BC\}$ ambas adicionadas por B, removemos AB que é chave mas tem A dentro de si então não é minimal e BC que não é chave e adiciona ABC na lista, removemos o ABC que também é chave mas tem A dentro e portanto não é minimal.

A lista está vazia e portanto o loop acabou, temos então apenas A como chave candidata.

4) *BCNF e 3FN*: Nesse caso não há violações em BCNF e nem em 3FN, já que A é superchave e todos os atributos do lado direito das DFs são primos.

VIII. LIMITAÇÕES

A entrada das DFs devem ter no máximo 1000 caracteres.